



南開大學
Nankai University

计算机学院
计算机科学与技术

软件工程团队作业
智能城市生成系统软件开发报告

小组成员

吴红祥 (2312558)、周昊川 (2310126)、吴行健
(2310686)

朱晨瑞 (2312674)、王旭 (2312166)、岳科言
(2312163)、谭易 (2312260)

2026 年 6 月 14 日

目 录

一、	项目概述	1
	开发背景	1
	项目目标	1
	开发环境	2
	可行性分析	2
	项目计划	3
二、	需求分析与系统设计	5
	需求分析	5
	系统设计	8
	详细设计	10
	数据库设计	29
	UI 设计	29
三、	系统测试	34
	测试环境	34
	功能测试	35
	核心验收清单	37
	Blender 手动测试脚本	37
	性能测试	38
四、	项目管理	39
	参与人员及分工	39
	项目进展记录	40
	项目管理工具	42
五、	用户手册	42
	总结与展望	45

一、项目概述

1.1 开发背景

随着智慧城市、数字孪生与城市仿真需求增长，行业分析师与场景建模师需要在统一平台完成「需求提出 → 场景生成 → 效果验收 → 成果展示」。传统方案往往将 Web 管理、三维建模、自然语言交互与实时渲染割裂在不同工具中：分析师在文档与邮件中描述需求，建模师在 DCC 软件中手工搭建，验收结论难以与渲染结果一一对应，权限与审计也无法集中管控。课程项目希望在有限周期内验证一种更紧凑的协作形态。

SmartCityGen 智慧城市生成系统 因此构建 **Web 主系统 + Blender 插件 + 语义解析 + API 内嵌渲染桥接 + 资产库**一体化平台。用户无需在多个独立产品间切换账号与数据格式；中文指令在 Web 端即可预览为 JSON 命令并驱动后台 Blender 执行，渲染进度与预览帧通过 WebSocket 与 HTTP 回传。当前生产链路为：**前端 → API(render_bridge) → Blender Runtime Worker → WebSocket 实时预览**，下文将按背景、需求、设计、测试与使用说明展开。

1.2 项目目标

从交付物视角看，系统可划分为五个相互协作的子系统：Web 承担组织与权限，Blender 承担三维构造，语义层承担意图翻译，渲染桥接承担算力调度，资产库承担可复用配置。下表概括各子系统职责；表后文字进一步说明它们如何串联为可演示的完整链路。

表 一、.1: 子系统职责

子系统	职责
Web 主系统	登录、RBAC、插件审核、需求验收、成果展示
Blender 插件	场景生成、模板、NL 编辑、布局、渲染
语义解析	中文指令 → JSON 命令
API 渲染桥接	Worker 心跳、任务队列、帧回传、WebSocket
资产库	道路纹理、树木、路灯、车辆、模板配置

系统整体定位说明

表 1 从子系统维度概括了各模块的职责边界。需要强调的是，SmartCityGen 智慧城市生成系统 并非五个彼此独立的工具，而是一条面向智慧城市场景建模的**协同生产链路**：

Web 主系统承担「谁可以做什么、需求如何流转、成果如何验收」的组织职能；Blender 插件与资产库负责「三维场景如何被程序化构造与替换」；语义解析层把分析师与建模师的自然语言意图翻译成可执行的 JSON 命令；API 渲染桥接则把耗时的 Blender 渲染从 HTTP 请求中解耦出来，通过 Worker 心跳、任务队列与 WebSocket 把「后台算力」与「前台预览」连接起来。

这种划分的好处在于：前端不必等待 Blender 完成整帧渲染；建模师既可以在 Web 端快速验证 NL 指令效果，又可以在需要精细调整时打开与 Worker 同源的 .blend 文件，避免「Web 预览与本地编辑不是同一场景」的常见集成问题。

1.3 开发环境

操作系统：Windows 11

开发语言：JavaScript, Python

开发工具：Chrome, Visual Studio Code, Blender 5.1, Node.js LTS

本项目的开发环境以 Windows 11 为主，这与课程演示及多数成员本机条件一致。前端采用 JavaScript (React + Vite)，后端与 Worker 脚本采用 Python (Blender 内 bpy 与独立 runtime_worker)，二者通过 Node.js Express API 与 JSON 契约衔接。Chrome 用于 Web 联调，Visual Studio Code 作为统一 IDE，Blender 5.1 作为三维运行时。上述工具均为成熟生态，文档与社区资源丰富，有利于团队在有限周期内完成「Web 控制台 + 三维生成 + 实时预览」的完整闭环。

版本选择上，Node.js 采用 LTS 以保证 npm install 与 Express 依赖稳定；Blender 4.x/5.x 均可运行 icity，但需注意 bpy API 在 5.0 附近对动画模块有小幅变更，path_animation 已做兼容。团队统一在 start.ps1 中配置 Blender 绝对路径，避免「成员本机路径不同导致 Worker silently 失败」的联调问题。

1.4 可行性分析

1.4.1 技术可行性

本系统采用「Web 控制台 + Node.js API + Blender Python 插件」分层架构，各层技术成熟、社区文档完善，与课程实验及智慧城市场景生成目标匹配。

前端层：基于 React 18 与 Vite 构建单页应用，配合 React Router 实现三角色路由与菜单守卫；LlmRenderPanel 通过 HTTP 与 WebSocket 与后端交互，技术路线与主流 Web 管理平台一致。

服务层：Express 提供 REST 鉴权与业务接口，内嵌 render_bridge 维护任务队列、Worker 心跳与帧缓存，ws 模块推送实时预览，避免前端长时间阻塞等待 Blender 渲染。

三维与语义层：Blender bpy 支持道路、建筑、树木的程序化生成；icity 插件提供 `command_executor` 统一分发 NL 命令；API 层 `inferCommandsFromPrompt` 将中文关键词映射为符合 `command.schema.json` 的 JSON，降低 Web 与 Blender 之间的耦合。

集成可行性：`runtime_worker.py` 以无头 Blender 轮询 API 任务并回传 960×540 预览帧；建模师可通过 `open-linked-blender` 打开与 Worker 同源的.blend 文件，保证 Web 预览与本地编辑场景一致。`shared/contracts` 以 JSON Schema 约束跨模块命令与模板格式，便于前后端与插件协同演进。

综上，所选技术栈可在 Windows 11 本机通过 `deploy.ps1 / start.ps1` 一键部署并联调，技术路径清晰、实现边界明确。

1.4.2 操作可行性

角色与流程：系统按 `admin / analyst / modeler` 划分职责，分析师负责需求与验收、建模师负责场景生成与渲染、管理员负责审核与审计，与智慧城市项目中「需求—建模—验收」协作流程一致，用户学习成本集中在各自工作台内。

部署与使用：`scripts` 目录提供 PowerShell 一键脚本，降低环境配置门槛；演示账号与 RBAC 菜单由 API 统一下发，新成员可按角色快速进入对应页面。Blender 插件通过偏好设置安装，Web 端 NL 输入与 JSON 预览降低对 bpy 脚本的直接操作要求。

1.5 项目计划

项目按模块化分工推进，主要阶段如下：

表 一、.2: 项目阶段划分

阶段	主要工作
P0	项目骨架、共享契约、API/Web/Blender 插件目录与一键脚本
P1	登录与 RBAC、渲染桥接、 <code>runtime_worker</code> 任务回传与 Web 实时预览
P2	icity 插件能力整合、功能测试与验收文档

表 2 将开发过程划分为 P0-P2 三个阶段，体现「先打通骨架与主链路，再丰富插件能力，最后以测试与文档收口」的递进策略。P0 阶段解决的是「能否一键部署、各目录与契约是否就位」；P1 阶段聚焦课程验收所依赖的核心能力——登录与 RBAC、渲染桥接、Worker 回传与 Web 实时预览；P2 阶段在 icity 插件能力稳定后，通过功能测试与验收文

档把「可演示」提升为「可说明、可复现」。各阶段并非严格瀑布，而是在共享契约约束下并行推进，但里程碑顺序保证了联调时始终有一条可运行的主路径。

阶段之间的协作方式为：契约先行——凡涉及 command、template、RBAC 的变更，须先更新 shared/contracts/ 再改 API 与插件；联调以 P1 主链路为每日冒烟，P2 能力增量通过 NL 回归集与 Blender 手动脚本验证。文档与报告与代码同仓库维护，避免「实现已变、说明滞后」的答辩风险。

当前实现完成度

除阶段计划外，表一、.3 从仓库模块维度汇总当前已交付能力，便于对照「设计目标」与「实际代码」是否一致。阅读时可结合 P0-P2 里程碑理解各模块在整体链路中的位置。

表一、.3: 模块已完成工作

模块路径	已实现能力
services/api	登录、RBAC、插件审核、需求/验收、规则 NL 解析、render_bridge、WebSocket、Worker 接入、打开同源 Blender
web/apps/main-console	三角色页面、LlmRenderPanel、实时预览、路由守卫、API 驱动菜单
blender-plugin/runtime_worker.py	心跳、拉任务、apply_commands、960×540 渲染、live-frame 回传
blender-plugin/smart_city_generator	面板插件 v0.2.0、模板服务、NL 演示、Worker 联调
blender-plugin/addons/icity	基础城市生成、command_executor、path_animation、动态元素与布局控制
blender-plugin/addons/nlp_city_editor	NL 场景编辑器 v2.0.0、24 类场景驱动函数
shared/contracts	command/template/rbac JSON Schema 契约
scripts/deploy.ps1, start.ps1	一键部署依赖、启动 API+Web+Worker
report/	软件开发报告 LaTeX 源稿与编译脚本

仓库目录结构

```
SmartCityGen/
+- docs/                # 架构、测试计划
+- report/              # 软件开发报告 (LaTeX)
+- shared/contracts/    # 跨模块 JSON 契约
+- web/apps/main-console/ # React 主控制台
+- services/api/        # 主 API + 内嵌渲染桥接
+- blender-plugin/
| +- runtime_worker.py  # 生产 Worker (直连 API)
| +- smart_city_generator/ # 精简可安装插件
| +- addons/icity/      # 城市生成插件
| +- addons/nlp_city_editor/ # NL 场景编辑器
+- scripts/             # deploy.ps1, start.ps1
```

仓库采用 monorepo 布局: `shared/contracts` 置于根目录, 保证 Web、API 与 Blender 插件引用同一份 JSON Schema; `services/api` 与 `web/apps/main-console` 分别独立 `package.json`, 便于分别启动与调试; `blender-plugin` 下同时保留生产 Worker、可安装面板插件与 `icity` 完整插件, 满足不同演示场景。报告源稿与编译脚本位于 `report/`, 与代码同步迭代。

二、需求分析与系统设计

2.1 需求分析

智慧城市生成平台的用户并非单一角色, 而是围绕「需求提出—场景实现—成果验收」形成协作链条。系统管理员 (admin) 负责平台治理: 审核 Blender 插件是否可接入、查看操作日志, 保证环境可控可追溯。行业分析师 (analyst) 代表业务侧: 提交建模需求、依据功能清单组织验收、查看渲染结果以确认 NL 指令是否满足预期。场景建模师 (modeler) 负责三维实现: 查阅模板与任务、通过 NL 或面板驱动场景生成、提交渲染并可在必要时打开与 Worker 同源的 Blender 做本地精修。

上述角色划分直接决定了功能性需求的优先级与界面布局——管理员不进入渲染面板, 分析师可预览但不可打开 Blender, 建模师拥有完整的生成与编辑入口。非功能需求则从运行体验与部署成本出发: 实时预览帧率可配置 (默认 20 FPS), 在流畅度与算力之间折中; API 采用 Bearer 鉴权, 配合 RBAC 限制接口与菜单; Worker 假定在本机信任域接入, 符合课程演示的一机部署模式; 一键脚本降低环境搭建门槛; Blender 4.x+ 与 Chrome 则与目标用户环境保持一致。

下表将功能性需求编号化，便于与测试用例、验收清单逐项对照；阅读时可将每条需求理解为「用户故事 + 验收条件」的压缩表述。

功能性需求详表

课程验收要求将「登录与权限、插件审核、需求与验收、NL 渲染、实时预览、同源 Blender」等行为具体化。下表按 Web 与 Blender/NL 两条线列出编号化需求；「优先级」P0 表示验收必需，P1 表示增强体验或建模师专属能力。实现位置列便于开发联调时快速定位源码。

Web 主系统需求

表 二、.1: Web 功能性需求

编号	需求描述	优先级	实现位置
F-W01	账号密码登录，按角色跳转工作台	P0	LoginPage, auth.js
F-W02	RBAC 菜单过滤与路由守卫	P0	ProtectedRoute, menus.js
F-W03	管理员审核插件 (pending/approved/rejected)	P0	AdminPluginReview
F-W04	分析师提交需求、勾选验收、保存结论	P0	AnalystRequirements, Acceptance
F-W05	建模师查看模板配置与渲染结果页	P0	ModelerTemplates, RenderResults
F-W06	LLM 面板: NL → JSON → 提交渲染	P0	LlmRenderPanel
F-W07	实时渲染预览 (WebSocket + live-frame)	P0	LlmRenderPanel, render_bridge
F-W08	建模师打开与实时流同源的 Blender	P1	open-linked-blender API
F-W09	系统操作日志查询	P1	AdminLogs, /api/logs

Blender 与语义解析需求

表 二、.2: Blender 与 NL 功能性需求

编号	需求描述	优先级	实现位置
F-B01	基础城市生成 (道路、建筑、树木)	P0	icity/generate_base
F-B02	模板 0/1 差异化生成	P0	template_service, apply_template
F-B03	≥5 条中文 NL 指令稳定执行	P0	core.js + icity/nl_handlers
F-B04	道路纹理替换	P0	icity/road_texture
F-B05	路灯/车辆/船只资产加载	P0	street_lights, dynamic_elements
F-B06	动态元素路径动画	P1	path_animation.py, Follow Path
F-B07	生态元素 (水体/山体)	P1	scene_enhance, landscape
F-B08	渲染输出图片/视频	P0	runtime_worker, render_output
F-B09	Worker 拉取任务并回传帧	P0	runtime_worker.py
F-N01	中文指令 → JSON 命令	P0	inferCommands-FromPrompt
F-N02	符合 command.schema.json	P0	shared/contracts
F-N03	环境/天气/路灯/动态/道路/模板动作	P0	契约枚举 + 执行器

Web 需求 (F-W 系列) 侧重平台协作与渲染链路, Blender/NL 需求 (F-B、F-N 系列) 侧重三维能力与命令语义。测试章节中的用例编号、验收清单与上表 F-xx 编号可逐项追溯; 优先级 P0 项为课程答辩演示必需, P1 项为体验增强, 未通过 P0 项视为版本不可验收。

2.2 系统设计

系统设计的目标，是在不增加用户使用复杂度的前提下，把 Web 管理、自然语言交互、异步渲染与 Blender 场景编辑串联为一条可重复演示的生产链路。为此我们从权限、指令、资产三条主线描述系统行为；每条主线对应一类「从用户动作到系统响应」的完整路径，而非孤立模块列表。

权限流：用户登录后，API 根据账号角色加载权限集合，并据此过滤可展示的侧边栏菜单；前端路由守卫在页面跳转时二次校验，防止通过 URL 越权访问。建模师在渲染结果页还可触发「打开同源 Blender」，该能力仅在其角色与权限同时满足时可用，保证 Web 预览与本地编辑指向同一场景文件。

指令流：用户在 LlmRenderPanel 输入中文描述，API 的 `inferCommandsFromPrompt` 将其映射为符合契约的 JSON 命令包；提交渲染后，`render_bridge` 创建 `queued` 状态任务，本机 Worker 轮询认领并在 Blender 中 `apply_commands`；渲染帧与进度经 HTTP/WebSocket 回传，Web 端更新预览。该链路异步化，避免 HTTP 长时间阻塞。

资产流：模板编号对应 `templates.json` 中的树木类型、道路纹理、建筑密度、天气与路灯等参数；插件加载资产并刷新场景后，模板 0 与模板 1 呈现可对比的日/夜与密度差异，支撑验收中的「模板化生成」项。

在权限、指令、资产三条主线之外，系统还包含渲染桥接、RBAC 矩阵与 `icity` 插件模块等实现细节。`render_bridge` 是 Web 与 Blender 之间的「调度中枢」；RBAC 矩阵定义三类角色可执行的权限集合；`icity` 插件则在 Blender 进程内完成实际的场景构造与命令执行。以下分节说明各组件职责及模块划分。

渲染桥接设计

内嵌于 `services/api/src/services/render_bridge.js` 的渲染桥接负责：

- **任务队列：**内存 Map 维护 `RenderJob`，状态为 `queued/running/completed/failed`；
- **Worker 注册：**心跳写入 `workerId`、`pid`、`blendPath`、`scene`、`online`；
- **帧缓存：**任务帧 (`frame`) 与实时预览帧 (`live-frame`) 以 base64 PNG 缓存；
- **WebSocket：**路径 `/render/stream/:jobId`，推送 `job_update`；
- **Worker 拉取：**GET `/local/render/next?workerId=` 认领排队任务。

与 `services/api` 内嵌桥接在同一进程部署 API 与渲染桥接，支持 `live-frame`、`blend-Path` 心跳字段及 Web 联调。选择单进程内嵌而非独立渲染服务，是为降低课程演示环境下的进程编排成本；Worker 仍通过 HTTP 轮询与 WebSocket 订阅与桥接交互，未来若需横向扩展，可将 `render_bridge` 抽离为独立服务而不改变对外协议。

RBAC 权限矩阵

权限设计遵循「最小够用」原则：分析师可提交需求、组织验收并预览渲染，但不具备打开本地 Blender 的能力，避免业务侧误改三维场景；建模师拥有 scene.generate 与完整渲染链路，可在 Web 验证 NL 后再进入 Blender 精修；管理员聚焦审核与审计，不介入具体建模操作。下表列出各角色对应的权限标识，与 rbac-matrix.json 及前端菜单过滤逻辑一致。

表 二、.3: 角色权限 (rbac-matrix.json)

角色	权限标识
admin	auth.login, rbac.manage, plugin.review, system.logs.view, plugin.access.manage, acceptance.view
analyst	auth.login, industry.requirements.submit, acceptance.review, acceptance.records.export, scene.render, llm.command.generate
modeler	auth.login, task.manage, template.view, scene.generate, scene.render, llm.command.generate

icity 插件概述

icity 为本项目采用的 Blender 城市生成插件,源码位于 blender-plugin/addons/icity/。插件通过 N 面板与 command_executor 统一分发 NL 命令,覆盖基础城市程序化生成、模板与道路纹理、路灯/车辆/船只等资产、路径动画、布局控制及场景增强;runtime_worker.py 在后台无头 Blender 中加载 icity 执行 Web 下发的渲染任务。

icity 核心模块

表 二、.4: icity 核心 Python 模块

模块	职责
<code>city_generator.py</code>	基础城市程序化生成
<code>command_executor.py</code>	统一 NL 命令分发
<code>nl_handlers.py</code>	规则 NL 命令 handler 注册与分发
<code>nl_intent_engine.py</code>	中文意图解析引擎
<code>nl_llm_backend.py</code>	LLM 后端适配
<code>assets_manager.py</code>	贴图与模型资产加载
<code>dynamic_elements.py</code>	车辆/行人/船只实例化
<code>path_animation.py</code>	路径曲线、Follow Path、动画烘焙
<code>template_manager.py</code>	模板配置读取与应用
<code>scene_enhance.py</code>	景观环绕、山体等生态增强

上述模块并非孤立文件，而是通过 `command_executor` 形成调用链：NL 相关请求经 `nl_intent_engine` 或 `nl_llm_backend` 解析后，由 `nl_handlers` 注册表分发至具体 handler，最终调用 `city_generator`、`template_manager`、`dynamic_elements` 等完成场景变更。`path_animation` 与 `scene_enhance` 则在基础城市生成之后叠加动态与生态能力，支撑验收清单中的 P1 项。

2.3 详细设计

本节在架构描述之下展开实现级说明，涵盖 API 端点、后端渲染桥接、Blender 插件、共享契约及 UML 设计图。建议先建立「异步渲染闭环」的整体印象，再查阅各端点与模块表；表格提供快速索引，各小节引导段说明设计动机与模块协作关系。

Express 路由层对外提供 REST 接口，对内调用 `store.js` 与 `render_bridge.js`。除健康检查与 Worker 本机端点外，业务接口均经 Bearer 鉴权与 RBAC 校验。端点设计遵循「资源 + 动作」习惯：认证与菜单、插件与需求、LLM 解析、渲染任务、日志审计分组清晰，便于前端 `client.js` 封装与权限守卫对齐。

API 端点详表

表 二、.5: API 端点一览

路径	方法	鉴权	说明
/health	GET	无	API 与桥接健康检查
/api/auth/login	POST	无	登录返回 Token
/api/auth/profile	GET	Bearer	用户信息与权限
/api/rbac/menus	GET	Bearer	角色侧边栏菜单
/api/stats	GET	rbac.manage	管理员统计
/api/plugins	GET	Bearer	插件列表
/api/plugins/:id/review	POST	plugin.review	审核插件
/api/requirements	GET/POST	Bearer/提交权	需求列表与提交
/api/acceptance/result		POST	acceptance.review
/api/acceptance/records	GET	Bearer	验收历史
/api/templates	GET	Bearer	模板 JSON
/api/llm/generate-json	POST	llm.command.generate	规则 NL → JSON
/api/render/apply-command	POST		scene.render
/api/render/jobs/:id	GET	scene.render	查询任务状态
/api/render/live-frame	GET	scene.render	最新实时预览帧
/api/render/workers	GET	scene.render	Worker 列表
/api/render/open-linked-blender	POST	scene.render+modeler	打开交互 Blender
/api/logs	GET/POST		查看/登录

Worker 无鉴权端点说明: /local/blender/heartbeat、/local/render/next、/local/blender/frame、live-frame、status 等路径供本机 Worker 调用, 假定运行在信任域内, 故未要求 Bearer Token。该设计简化 Worker 与 API 的启动顺序, 但生产环境应通过网络隔离或 mTLS 限制

访问来源。

NL 规则解析详表

API 层 `inferCommandsFromPrompt` 当前映射规则如下（关键词规则引擎）：

表 二、.6: NL 关键词映射

关键词	action	params 示例
变暗 / 夜 / 天色暗	set_environment	mode=dark, brightness=0.25
雨	set_weather	mode=rainy
路灯	add_street_lights	count=12
车辆 / 汽车	add_dynamic_element	type=car, count=3
行人	add_dynamic_element	type=pedestrian
船	add_dynamic_element	type=boat
道路类型 1/2	set_road_texture	type=1/2
模板 0/1	apply_template	templateId=0/1
无匹配	apply_asset_config	默认树木/道路/座椅

runtime_worker 环境变量

表 二、.7: Worker 环境变量

变量	默认	说明
API_BASE_URL	localhost:3001	API 根地址
BLENDER_WORKER_ID	blender-runtime-worker	Worker 标识
BLENDER_LIVE_FPS	60	实时帧率 (start.ps1 可设 20)
BLENDER_HEARTBEAT_INTERVAL	2.0	心跳间隔 (秒)
BLENDER_INTERACTIVE_MODE	0	1 为 GUI 定时器模式

后端与渲染桥接详细设计

API 进程同时承载业务逻辑与渲染桥接，避免演示环境下多进程部署复杂度。认证模块负责 Token 生命周期；render_bridge 维护任务状态机与 WebSocket 订阅；store 提供演示用内存数据。理解「创建任务 → Worker 认领 → 回写帧 → 推送 job_update」的时序，是排查「任务卡住」类问题的关键。

认证机制

API 采用轻量级 Token 认证（演示环境，非 JWT）。登录成功后 auth.js 将 {userId} 序列化为 JSON 并经 base64url 编码返回；前端存入 localStorage 键 scg_token、scg_user。

authRequired 中间件解析 Bearer Token，在 users 数组中匹配 userId，注入 req.user。RBAC 由 rbac.js 读取 shared/contracts/rbac-matrix.json，requirePermission(perm) 校验角色是否包含权限标识。

render_bridge 任务生命周期

渲染桥接模块（render_bridge.js）在 API 进程内维护四个核心 Map：

- **jobs:** 渲染任务，含 id、prompt、commands、status、progress、previewUrl、claimedBy；
- **workers:** Worker 心跳快照，含 workerId、pid、appPath、blendPath、scene、online；

- **liveFrames**: 各 Worker 最新 live-frame;
- **subscribers**: 每个 jobId 对应的 WebSocket 客户端集合。

创建任务: createRenderJob 生成唯一 jobId, 初始 status=queued、progress=0, 写入 jobs Map。

Worker 认领: GET /local/render/next 按 FIFO 选取 queued 任务, 设置 claimedBy 与 status=running, 广播 job_update。

进度回写: POST /local/blender/status 更新 progress; POST /local/blender/frame 写入 base64 PNG 至 previewUrl, completed 时 progress=100。

实时预览: Worker 空闲时周期性 POST /local/blender/live-frame; 前端 GET /api/render/live 返回 latestLiveFrameWorkerId 对应帧。

占位预览: 未收到真实帧时, createPreviewSvg 生成 SVG data URL, 显示 Job ID、用户、Prompt、Actions 与进度条, 便于无 Blender 环境下的 UI 调试 (LOCAL_AGENT_AUTO_SIMULATE 可启用模拟进度)。

WebSocket 消息格式

连接路径: /render/stream/:jobId 或 /local/render/stream/:jobId。

服务端推送 JSON:

```
{
  "type": "job_update",
  "job": {
    "id", "status", "progress", "prompt",
    "commands", "previewUrl", "claimedBy", ...
  }
}
```

连接时若任务不存在, 推送 job_not_found。前端 LlmRenderPanel 在 onmessage 中更新 renderJob 状态, 并在 completed/failed 时关闭 WebSocket。

打开同源 Blender

POST /api/render/open-linked-blender 仅限 modeler 角色。服务端从 workers Map 读取目标 Worker 的 blendPath 与 appPath, 使用 Node.js child_process.spawn 启动交互式 Blender 进程, 加载与后台 Worker 相同的.blend 文件, 实现 Web 预览与本地编辑场景一致。

API 服务文件结构

表 二、.8: services/api 源码结构

文件	职责
src/server.js	Express 入口、挂载路由、WebSocket 服务
src/routes/auth.js	登录、profile
src/routes/core.js	业务 API、inferCommandsFromPrompt、打开 Blender
src/middleware/auth.js	Token 编解码与鉴权
src/middleware/rbac.js	权限校验
src/services/render_bridge.js	任务队列、Worker、帧缓存、广播
src/data/store.js	内存用户、插件、日志、需求、模板
src/utils/menus.js	菜单定义与按角色过滤

初始种子数据

store.js 预置三演示用户（admin/analyst/modeler）、插件状态 pending、一条示例需求、系统初始化日志。插件审核通过后建模师工作台可展示插件 approved 状态。重启 API 后除模板 JSON 外业务数据恢复为种子状态。

Blender 插件与操作符详表

Web 下发的 JSON 命令最终由 icity 插件内的 `command_executor` 分发到各 handler 与 bpy 操作符。插件层承担「把抽象 action 转为具体场景变更」的职责，包括基础城市生成、模板与纹理、动态元素、路径动画及 NL 执行入口。操作符按功能分类如下，完整列表以源码为准。

icity 操作符分类

表 二、.9: icity 操作符（按功能分类）

分类	bl_idname（代表）
基础与模板	scg.generate_base_city, scg.apply_template, scg.apply_road_texture
资产与灯光	scg.apply_street_lights, scg.add_landscape_surround
动态元素	scg.add_dynamic_elements, scg.add_boats, scg.export_manual_paths
布局控制	scg.fill_layout_demo, scg.apply_layout, scg.extract_layout_from_image
NL 与桥接	scg.execute_nl, scg.start_icity, scg.edit_city

完整操作符列表见 addons/icity/operators/ 下各模块源码。

nl_handlers 命令处理

command_executor 按 action 分发至 nl_handlers.py 中注册的 handler:

- apply_template → template_manager.apply_template
- apply_asset_config → 树木/道路/座椅/密度批量设置
- set_road_texture → city_generator.set_road_texture
- set_environment → 日/夜环境切换
- set_weather → 雨天环境切换
- add_dynamic_element → add_cars_and_pedestrians / add_boats
- add_street_lights → apply_street_lights

执行前调用 icity_scene_ready() 校验，未生成基础城市则提示「请先 Start ICity 并生成基础城市」。

path_animation 路径动画

core/path_animation.py（约 1800+ 行）实现阶段三动态元素核心能力：

- 从道路网格提取路径曲线 (SCG_Path_ 前缀、SCG_Paths 集合);
- 车辆/行人沿道路边缘 Follow Path 关键帧动画;
- 船只沿水体开阔水域环线 (SCG_Path_Boat_Loop), 避让城市边界与岸距;
- 兼容 Blender 5.0+ 动画 API (anim_utils channelbag);
- 支持手动路径资产目录 assets/manual_paths/。

runtime_worker 主循环

blender-plugin/runtime_worker.py 为当前 Web 联调生产路径:

1. main_loop_background: bpy.app.timers 或 while 循环;
2. 定时 heartbeat → blendPath/scene 回写;
3. GET /local/render/next 认领任务;
4. apply_commands + render_to_temp (960×540);
5. 上传 frame/status; 空闲时上传 live-frame;
6. 自动保存.blend 至临时工作区 (ensure_blend_path)。

交互模式 (BLENDER_INTERACTIVE_MODE=1) 使用 main_loop_interactive 定时器, 便于与 GUI Blender 共存调试。

综上, Blender 侧的设计核心是「命令可组合、执行可观测」: Web 只需下发符合契约的 JSON, 插件层负责将其翻译为一系列 bpy 操作; Worker 则负责在无人值守环境下循环执行上述流程并回传可视化结果, 使分析师与建模师在浏览器中即可感知场景变化。

共享契约与非功能性设计

为避免 Web、API、Blender 三端各自定义命令格式, 项目在 shared/contracts/ 中用 JSON Schema 约束 command 包与 template 配置。契约是联调的「单一真相源」: API 的 inferCommandsFromPrompt 输出、前端 JSON 预览、Worker 的 apply_commands 输入均须通过同一 schema 校验。非功能性设计 (性能、安全、可靠性) 则在契约之下补充运行约束与演示环境局限说明。

command.schema.json 命令协议

命令包为 JSON 对象, **必填字段**: source、raw_text、commands。

- **source** 枚举: nl_input (自然语言)、template (模板按钮)、button (面板按钮);

- **raw_text**: 原始用户输入，minLength=1;
- **commands**: 动作数组，每项含 action + params（开放对象）。

action 枚举(契约层): set_environment、set_weather、add_street_lights、add_dynamic_element、set_road_texture、apply_asset_config、apply_template、build_layout。

示例:

```
{
  "source": "nl_input",
  "raw_text": "将天色变暗并添加车辆",
  "commands": [
    { "action": "set_environment",
      "params": { "mode": "dark", "brightness": 0.25 } },
    { "action": "add_dynamic_element",
      "params": { "type": "car", "count": 3 } }
  ]
}
```

template.schema.json 模板协议

模板键为数字字符串（"0"、"1"...）。**必填**: tree_type、road_texture、bench_type、building_density（low/medium/high）、weather（day/night/rainy）。**可选**: street_light（boolean）。

当前 templates.json 数据:

表 二、.10: 模板 0 与模板 1 配置

ID	tree	road	bench	density	weather	light
0	1	2	1	medium	day	—
1	2	1	2	high	night	true

性能设计

- 前端交互与耗时渲染解耦: apply-command 立即返回 jobId，不阻塞 HTTP;
- 任务进度通过 WebSocket 推送 + HTTP 轮询双通道保障;
- live-frame 与 BLENDER_LIVE_FPS 联动，start.ps1 全局默认 20 FPS 降低 CPU 占用;

- Blender 单帧分辨率固定 960×540，平衡预览清晰度与渲染耗时；
- Worker 心跳 2s，任务轮询间隔由 FPS 推导，避免空转占满 CPU。

安全性设计

- **API 鉴权**：除 /health、/local/*、登录外，业务接口需 Bearer Token；
- **Worker 无鉴权**：假定本机信任域，仅开发/演示使用，生产需 IP 白名单或 mTLS；
- **角色隔离**：打开 Blender、插件审核、系统日志等按 permission 硬校验；
- **演示密码明文**：store.js 明文存储，生产必须哈希 (bcrypt) 并 HTTPS 传输；
- **操作审计**：plugin_review、acceptance、llm_generate、render_apply 等写入 logs。

可靠性与可扩展性

- 任务状态机明确 (queued/running/completed/failed)，失败保留 errorMessage 可重试；
- NL 解析与 Blender 执行分离，契约 JSON Schema 作为跨模块单一真相源，Web、API、Blender 共同遵守。

为弥补纯表格在「角色协作」与「流程时序」上的不足，本节给出用例图、活动图、类图与部署相关视图 (TikZ 绘制，不依赖外部图片)。用例图强调三类参与者与系统边界的对应关系；活动图描述从 NL 输入到预览完成的异步路径；类图与组件图则帮助理解 API、桥接与插件之间的静态结构。阅读时可将前文「指令流」与活动图对照，将「权限流」与用例图对照。

用例图

用例图描述系统边界、三类参与者与 SmartCityGen 提供的主要服务。场景建模师侧重场景生成与渲染；行业分析师侧重需求提交、功能验收与渲染预览；系统管理员侧重插件审核与系统日志。

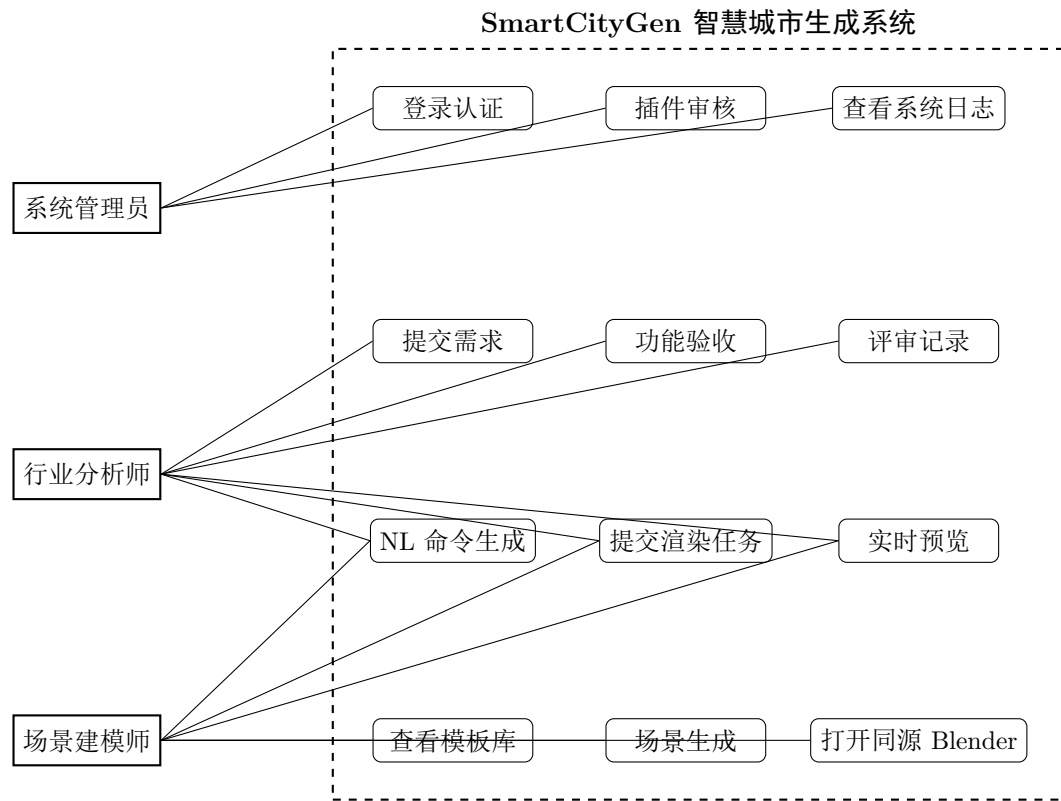


图 二、.1: SmartCityGen 用例图

活动图

活动图描述从自然语言输入到 Web 端实时预览的完整业务流程，体现「解析—编排—执行—回传」的异步闭环。

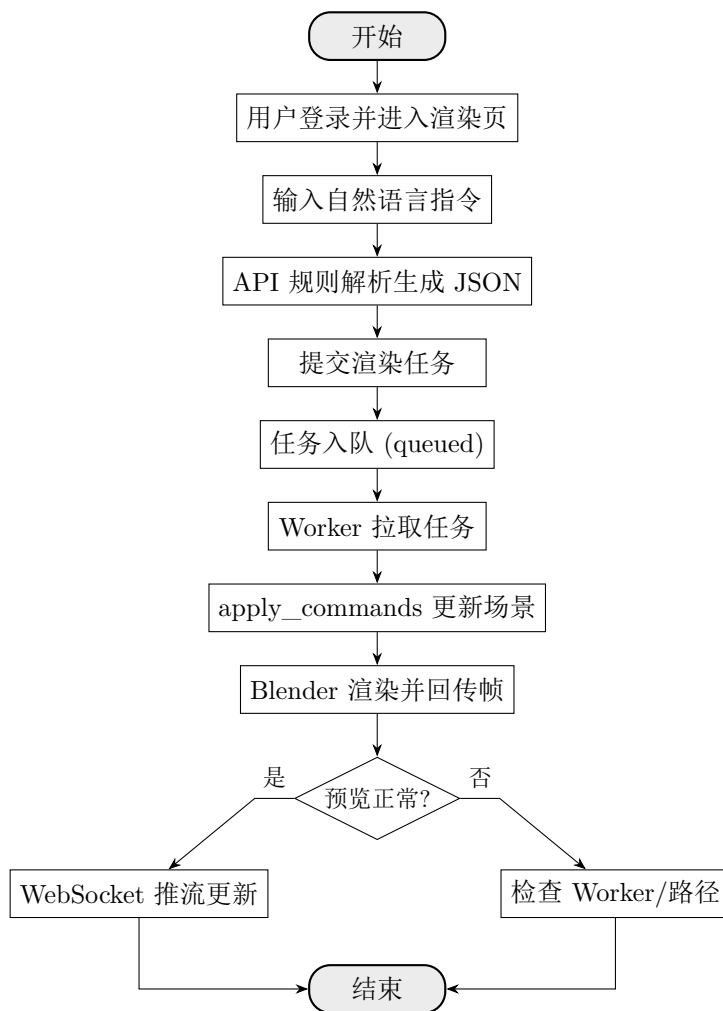
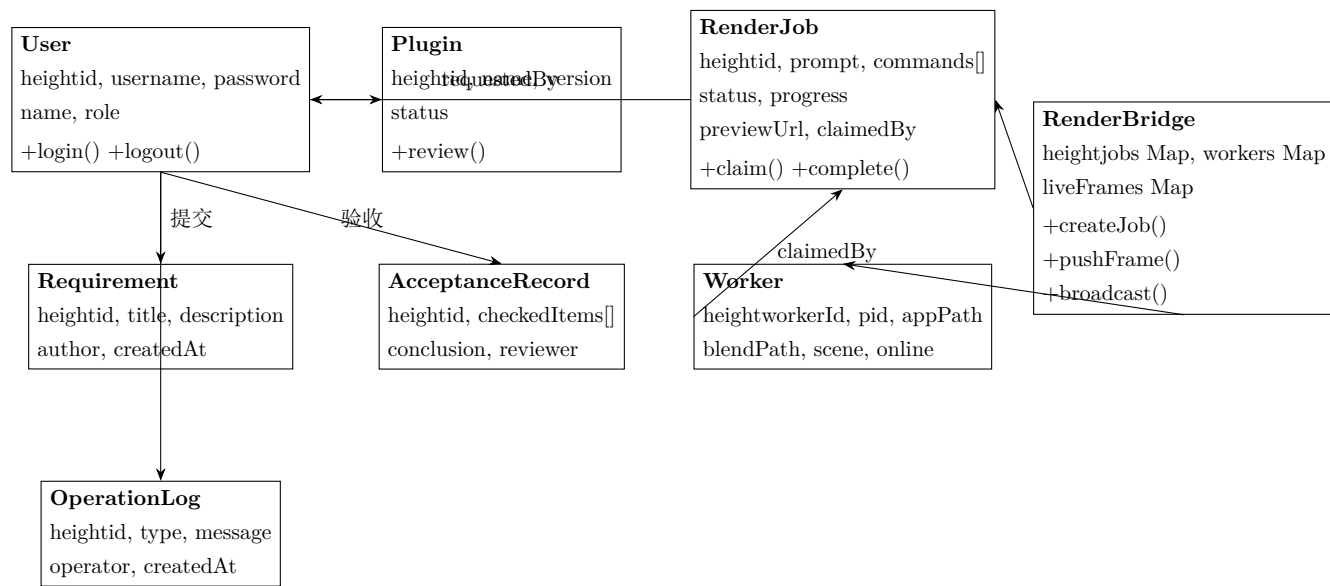


图 二、.2: 自然语言驱动渲染活动图

类图

类图以用户、业务实体与渲染桥接为核心，反映内存数据模型与运行时任务对象之间的关系。



图二、.3: SmartCityGen 核心类图

顺序图

顺序图描述一次「自然语言生成并渲染城市场景」的典型交互：Web 前端经 API 创建任务，渲染桥接调度 Worker 执行并回传帧。

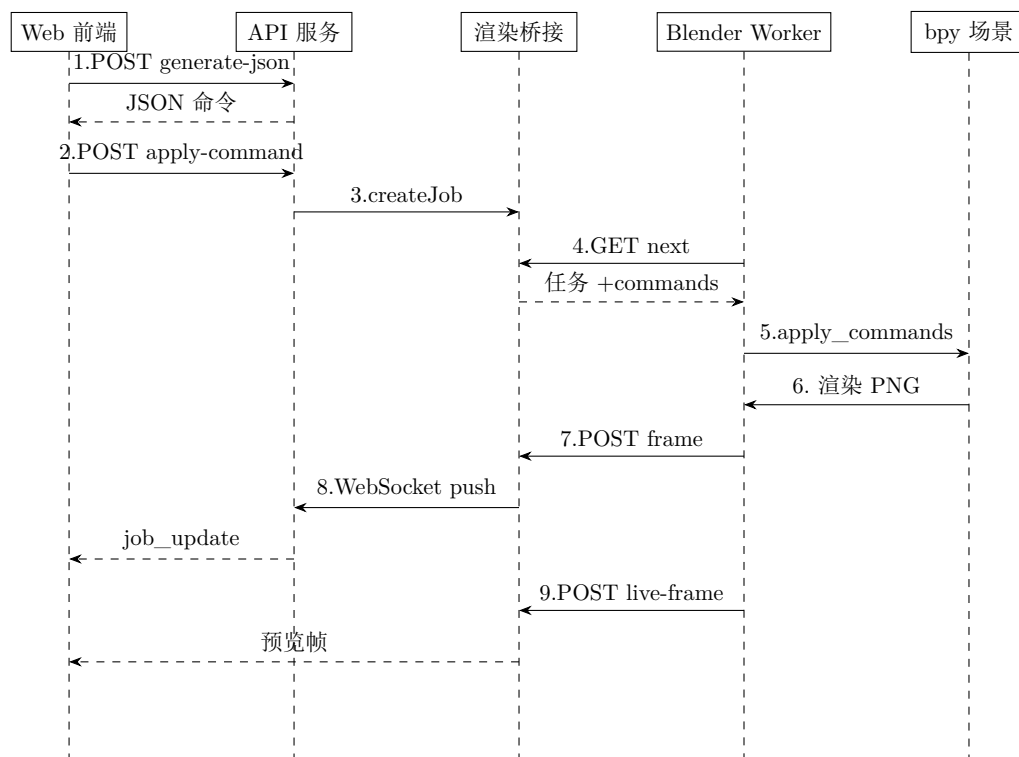


图 二、.4: 自然语言渲染顺序图

状态图

状态图描述渲染任务（RenderJob）在生命周期中的状态迁移，支撑前端进度展示与错误重试。

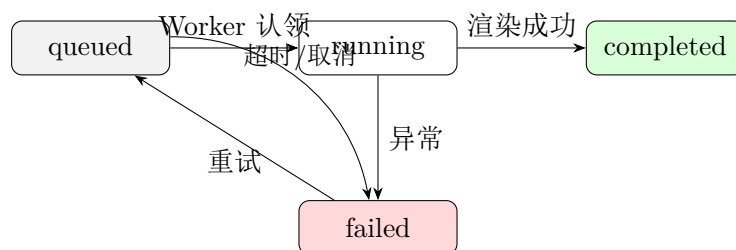


图 二、.5: 渲染任务状态图

构件图

构件图描述系统在实现层面的模块划分：Web 控制台、API 业务层、内嵌渲染桥接、runtime_worker 与 icity 插件之间的依赖关系。

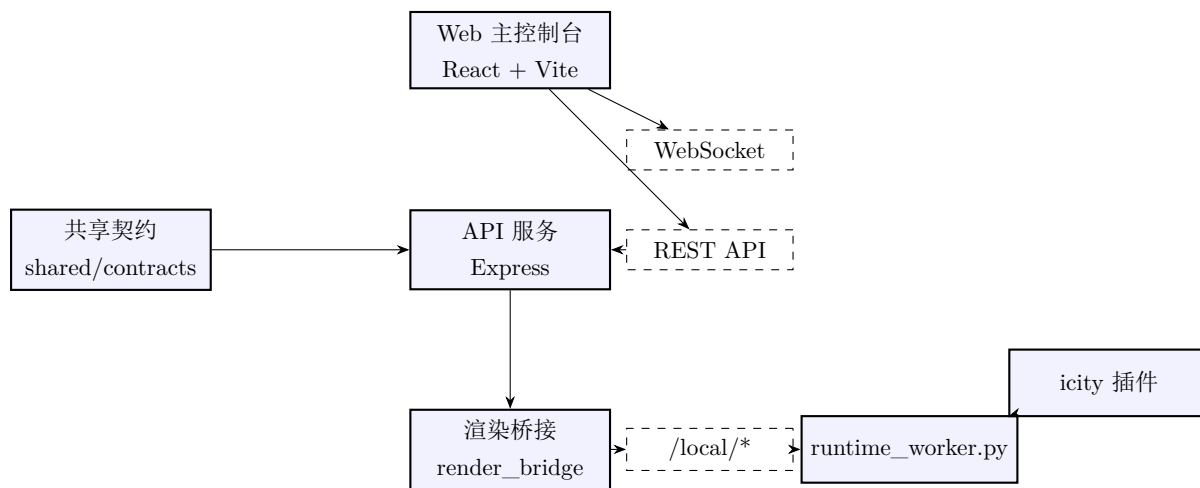


图 二、.6: SmartCityGen 构件图

部署图

部署图描述本地开发与演示环境的典型部署方式：用户浏览器访问 Web 前端，API 与渲染桥接同进程部署，Blender Worker 在本机后台运行。

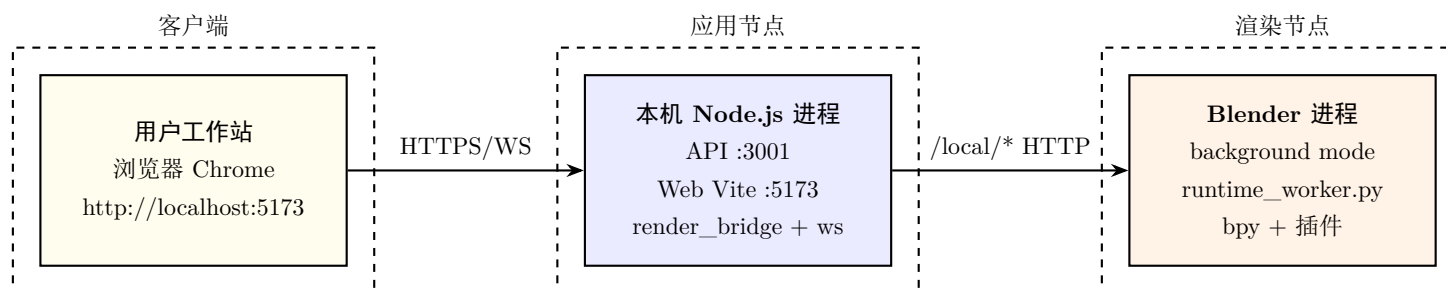


图 二、.7: SmartCityGen 本地部署图

协作图

协作图强调对象间的消息编号与结构关系，与顺序图互补。

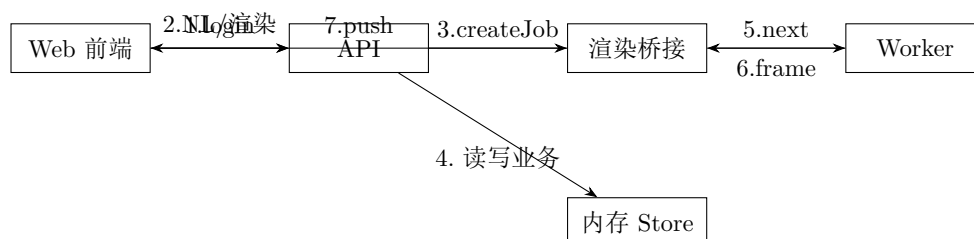


图 二、.8: 渲染链路协作图

2.4 数据库设计

数据层当前采用内存存储 (`store.js`)，这是演示阶段的简化方案：无需独立数据库即可完整演示登录、审核、需求、验收与审计；服务重启后恢复为预置种子数据，便于反复验收演示。尽管持久化方式简单，实体划分仍对应真实业务概念，渲染任务与 Worker 状态则由 `render_bridge` 单独维护，与业务 CRUD 解耦。核心实体包括：

- **users**：演示账号 `admin/analyst/modeler` 及角色权限；
- **plugin**：插件审核状态 (`pending/approved/rejected`)；
- **requirements**：行业分析师提交的需求条目；
- **acceptanceRecords**：功能验收勾选项与结论；
- **logs**：登录、审核、渲染等操作审计；
- **render_jobs / workers**：由 `render_bridge` 内存维护的任务队列与 Worker 心跳；
- **templates**：只读自 `templates.json`，模板 0 为 medium 密度、day 天气；模板 1 为 high 密度、night 天气并启用路灯。

模板配置以 JSON 文件为单一来源，Web 模板库页与 Blender `apply_template` 读取同一份数据，避免展示与执行不一致。若未来迁移至关系型数据库，用户、需求、验收与日志等实体可直接映射为表结构，渲染队列亦可改为持久化 Job 表，而契约层 JSON 格式无需变动。

2.5 UI 设计

界面设计以「角色工作台」为中心：不同用户登录后看到不同的侧边栏与默认 landing 页，减少无关功能干扰。菜单由 `API /api/rbac/menus` 按权限动态下发，前端 Layout 渲

染导航，`ProtectedRoute` 守卫路由。`LlmRenderPanel` 是分析师与建模师共享的核心组件，在同一页面完成 NL 输入、JSON 预览、任务提交、进度跟踪与实时帧展示；建模师另增「打开当前实时来源的 Blender」按钮，支持 Web 指令验证后的本地精细编辑。整体风格与主流 Web 管理平台一致，降低学习成本。

Web 菜单与路由

侧边栏菜单并非写死在前端，而是由 `/api/rbac/menus` 按当前用户权限动态返回，再与 React Router 路由绑定。这样管理员、分析师、建模师登录后看到的导航结构自然隔离，越权访问路径时 `ProtectedRoute` 会拦截。下表列出菜单标题、路径与所需权限的对应关系。

表 二、.11: Web 菜单定义

菜单标题	路径	所需权限
系统首页	<code>/admin/home</code>	<code>rbac.manage</code>
角色权限管理	<code>/admin/roles</code>	<code>rbac.manage</code>
插件审核	<code>/admin/plugin-review</code>	<code>plugin.review</code>
系统日志	<code>/admin/logs</code>	<code>system.logs.view</code>
行业需求提交	<code>/analyst/requirements</code>	<code>industry.requirements.submit</code>
功能验收	<code>/analyst/acceptance</code>	<code>acceptance.review</code>
评审记录	<code>/analyst/records</code>	<code>acceptance.records.export</code>
渲染结果 (分析师)	<code>/analyst/render-results</code>	<code>scene.render</code>
工作台首页	<code>/modeler/home</code>	<code>task.manage</code>
当前任务需求	<code>/modeler/tasks</code>	<code>task.manage</code>
模板库	<code>/modeler/templates</code>	<code>template.view</code>
渲染结果 (建模师)	<code>/modeler/render-results</code>	<code>scene.render</code>

Web 前端设计

主控制台位于 `web/apps/main-console`，采用 React 18 单页应用，路由由 React Router 6 管理，API 调用集中于 `src/api/client.js`。页面按角色划分目录，公共布局与 Llm-

RenderPanel 复用于分析师与建模师的渲染场景；各页面职责单一，避免在同一界面堆叠过多无关功能。下表汇总路由与组件映射，后文对 LlmRenderPanel 的交互流程作进一步说明。

页面与路由一览

表 二、.12: React 页面路由

路由	组件	角色	功能说明
/login	LoginPage	公开	账号密码登录，Token 写入 localStorage
/admin/home	AdminHome	admin	用户数、插件状态、日志数统计卡片
/admin/roles	AdminRoles	admin	用户角色表（UI 演示，无 API 持久化）
/admin/plugin-review	AdminPlugin-Review	admin	审核 icity 城市生成插件
/admin/logs	AdminLogs	admin	拉取 /api/logs 展示操作记录
/analyst/requirements	AnalystRequirements	analyst	需求列表与新建表单
/analyst/acceptance	AnalystAcceptance	analyst	7 项验收清单勾选与保存
/analyst/records	AnalystRecords	analyst	历史验收记录列表
/analyst/render-results	AnalystRender-Results	analyst	LlmRenderPanel（无打开 Blender）
/modeler/home	ModelerHome	modeler	插件状态、快捷入口
/modeler/tasks	ModelerTasks	modeler	本地 Mock 任务列表
/modeler/templates	ModelerTemplates	modeler	templates.json 格式化展示
/modeler/render-results	ModelerRender-Results	modeler	LlmRenderPanel + 打开 Blender

登录后根路径 / 按角色重定向:admin → /admin/home;analyst → /analyst/requirements;

modeler → /modeler/home。

公共组件

表 二、.13: 前端公共组件

组件	职责
Layout.jsx	侧边栏菜单（/api/rbac/menus）、退出登录
ProtectedRoute.jsx	校验 scg_token，按 role 守卫路由
LlmRenderPanel.jsx	NL 对话、JSON、渲染任务、实时帧、Worker 信息
Toast / Spinner / EmptyState	通用反馈与空状态
ConfirmDialog / ErrorBoundary	确认框与错误边界

LlmRenderPanel 交互设计

LlmRenderPanel 是分析师与建模师渲染页的核心组件，实现「自然语言 → 结构化命令 → 异步渲染 → 实时预览」闭环。

状态管理：input（用户输入）、messages（对话历史）、generatedJson、renderJob、liveFrame、workers、bridgeStatus。

关键流程：

1. 用户输入 NL 文本，点击「生成 JSON」→ POST /api/llm/generate-json;
2. 展示格式化 JSON，点击「提交渲染」→ POST /api/render/apply-command;
3. 建立 WebSocket: ws://localhost:3001/render/stream/:jobId, 接收 job_update;
4. 降级轮询：任务状态每 1.5s HTTP 轮询 /api/render/jobs/:id;
5. 实时帧:按 VITE_REALTIME_RENDER_FPS(默认 60,start.ps1 可设 20)轮询 /api/render/live-frame;
6. Worker 列表：每 3s 轮询 /api/render/workers，展示 workerId、blendPath、online;
7. 建模师专属:allowOpenLinkedBlender=true 时显示「打开当前实时来源的 Blender」, 调用 POST /api/render/open-linked-blender。

上述流程刻意保留「生成 JSON 与提交渲染」两步，使用户在驱动 Blender 前有机会核对命令语义，减少误操作导致的长时间无效渲染。WebSocket 与 HTTP 轮询并存，是为在 ws 连接不稳定时仍能更新任务进度；live-frame 与任务帧分通道获取，保证 Worker 空闲时预览区也能持续刷新。

预览优先级:liveFrame.previewUrl > renderJob.previewUrl。任务未完成时 render_bridge 可返回 SVG 占位预览（含进度条与 action 列表），真实 Blender 帧上传后自动替换。

分析师验收清单说明

AnalystAcceptance 页面内置 7 项验收勾选项（与课程验收维度对应）：

1. 支持道路生成
2. 支持建筑生成
3. 支持资产替换
4. 支持模板化生成
5. 支持自然语言编辑
6. 支持交通/人群动态元素
7. 支持生态元素

勾选 ≥ 5 项时结论为「通过」，否则「部分通过」，结果写入 acceptanceRecords 并可通过评审记录页查询。

典型业务场景

为便于理解前文表格中的模块划分，下面给出三类角色在一次完整演示中的典型协作路径。

场景一：需求到验收（分析师主导）。分析师登录后进入「行业需求提交」，填写智慧城市建模诉求；建模师在同一需求背景下于 Web 端或 Blender 中完成场景构造；分析师在「功能验收」页按道路、建筑、模板、NL、动态、生态等维度勾选，保存结论并导出评审记录。该路径体现系统对「业务侧可追溯验收」的支持，而非仅提供三维工具。

场景二：自然语言驱动渲染（分析师/建模师）。用户在 LlmRenderPanel 输入如「将天色变暗并添加车辆」的中文描述；API 规则引擎解析为 JSON 命令包并在页面预览；确认后提交渲染任务，Worker 在后台 Blender 中执行 `apply_commands`，WebSocket 与 live-frame 轮询共同驱动预览区更新。该路径是课程验收的核心演示链路，强调「语义 → 结构化命令 → 异步渲染 → 实时反馈」的闭环。

场景三：平台治理（管理员）。管理员审核 icity 插件是否 approved，查看系统日志中的登录、审核、渲染类事件，必要时调整角色对插件的访问策略。该路径保证演示环境可管可控，与生产系统中「插件准入 + 审计」的思路一致。

设计取舍说明

团队在实现中作出若干有意识取舍，读者在查阅后文详表时可对照理解。**内存存储**替代关系型数据库，是为降低演示部署成本，实体划分仍按真实业务建模，便于后续迁移。**规则 NL 引擎**替代真实大模型调用，是为保证离线演示稳定、输出可预期，契约层 action 枚举与 icity 执行器已预留 LLM 后端扩展点。**渲染桥接内嵌 API 进程**是为避免多服务编排，Worker 仍可通过 HTTP/WebSocket 与桥接解耦扩展。**960×540 预览分辨率**是在清晰度与本机算力之间的折中，正式输出可依赖 Blender 侧高分辨率渲染。

报告阅读建议

本报告采用「概述 → 需求 → 设计 → 测试 → 管理 → 手册」结构：第 1 章建立背景与可行性；第 2 章需求详表与系统设计、详细设计、UML 可对照阅读——若希望先建立整体印象，可先浏览「权限流/指令流/资产流」三段叙述与 UML 活动图，再查 API 与插件模块表；第 3 章测试与第 6 章用户手册适合动手验证；第 4 章项目管理说明团队分工与仓库边界。表格与清单用于快速索引，各节引导段说明设计动机与模块协作关系。

三、 系统测试

3.1 测试环境

测试环境与演示部署保持一致，以保证「报告描述」与「实际可运行」一致。硬件与软件配置如下：Windows 11;Blender 4.x/5.x;Chrome;Node.js LTS 18+。执行 scripts/deploy.ps1 安装依赖，配置 start.ps1 中 Blender 路径后执行 start.ps1，即可启动 API、Web 与 Worker 完整链路。

表 三、.1: 演示账号

账号	密码	角色
admin	admin123	管理员
analyst	analyst123	分析师
modeler	modeler123	建模师

3.2 功能测试

功能测试验证系统是否满足课程验收清单与需求详表中的 P0 项。测试在完整部署环境（API + Web + Worker）下执行，覆盖三角色协作、NL 解析、渲染状态流转、实时预览与同源 Blender 等主链路；每项结果需与预期一致，P0/P1 缺陷须留存复现步骤与截图至 tests/evidence/。

表 三、.2: 核心验收清单 (10 项)

#	验收项	状态
1-3	三角色登录；插件审核；打开同源 Blender	成功
4-6	基础生成；模板 0/1 差异；≥5 条 NL	成功
7-10	动态元素；生态；渲染输出；实时预览	成功

功能测试用例详表

功能测试在真实部署环境下手工执行，并按类别覆盖 Web、RBAC、业务流转、NL、渲染链路 与 Blender 插件。每条用例包含操作步骤与预期结果，与需求编号及验收清单形成追溯关系。NL 回归集单独列表，保证规则引擎在多种中文表述下输出符合契约的 action。

表 三、.3: 功能测试矩阵（摘要）

类别	样例	操作步骤	预期结果
Web 登录	三角色	/login	正确跳转与菜单
RBAC	跨角色路由	手动访问	路由守卫拦截
插件/需求/验收	审核与保存	各工作台	状态与记录更新
NL 指令	7+ 条样例	渲染面板	对应 action JSON
渲染链路	提交任务	生成 JSON 后提交	queued→completed
实时预览	Worker 在线	观察预览区	live-frame 更新
打开 Blender	modeler	渲染结果页	同源.blend 拉起

类别	样例	操作步骤	预期结果
Blender 插件	安装启用	偏好设置	N 面板与基础生成
动态/生态	动画与环绕	icity 面板	路径运动/山体水体

NL 指令回归测试集

表 三、.4: NL 指令与预期 action（共 7+ 条）

输入	预期 action	备注
将天色变暗	set_environment	brightness≈0.25
切换雨天模式	set_weather	mode=rainy
添加路灯	add_street_lights	count=12
添加车辆	add_dynamic_element	type=car
添加船只	add_dynamic_element	type=boat
道路切换类型 1	set_road_texture	type=1
应用模板 1	apply_template	templateId=1
随机无关句	apply_asset_config	默认 fallback

缺陷分级与证据要求

P0：无法登录、插件无法启用、渲染完全失败；P1：模板/NL/资产核心失败；P2：效果偏差可绕过；P3：文案样式。

每个 P0/P1 缺陷需附：复现步骤、report.log 或 API 日志片段、Web/Blender 截图，存入 tests/evidence/。

3.3 核心验收清单

表 三、.5: 验收清单 (对照 docs/test-plan.md)

#	验收项	状态
1	三角色登录并看到不同工作台	成功
2	管理员可审核插件并改变状态	成功
3	建模师打开同源 Blender 成功	成功
4	插件基础生成道路/建筑/树木	成功
5	模板 0 与 1 视觉效果差异明显	成功
6	≥ 5 条中文 NL 指令稳定触发	成功
7	至少一种动态元素沿路径移动	成功
8	至少一种生态元素可见	成功
9	成功输出渲染图片或视频	成功
10	Web 端实时预览帧正常更新	成功

3.4 Blender 手动测试脚本

表 三、.6: icity 插件手动测试脚本

脚本路径	用途
ad-dons/icity/docs/nl_p0_manual_test.py	P0 命令执行 (暗/日/道路/模板/资产/车)
ad-dons/icity/docs/nl_p1_manual_test.py	P1 离线 NL 测试
ad-dons/icity/docs/nl_p1_offline_test.py	离线规则引擎
ad-dons/icity/docs/nl_p2_llm_test.py	LLM 后端测试

证据留存目录：`tests/evidence/`（截图、短视频、操作日志）。

当前版本 10 项核心验收均已通过；NL 回归集对环境、天气、资产、模板等常见中文指令给出稳定 JSON 输出，渲染任务可完成 `queued` 至 `completed` 的状态迁移，Worker 在线时 `live-frame` 持续更新。测试过程中发现的 P0/P1 问题均已在联调阶段修复，相关截图与日志留存于 `tests/evidence/`，与 `docs/test-plan.md` 保持一致。

3.5 性能测试

性能测试在功能通过的前提下进行，衡量 API 响应、NL 解析、单帧渲染与 `live-frame` 等可量化指标是否达到设计目标，并观察 Worker 长稳运行与连续 NL 提交下的资源占用。下列指标为验收参考基线，实际数值随本机 CPU 与场景复杂度略有波动。

表 三、.7: 性能指标

指标	目标	说明
<code>/health</code>	< 200 ms	健康检查
<code>generate-json</code>	< 1 s	NL 解析
<code>live-frame</code>	20 FPS	BLENDER_LIVE_FPS
单帧渲染	< 10 s	960×540 PNG

性能测试场景

性能测试在功能通过的前提下进行，关注用户可感知的响应时间与系统可持续运行能力。测试场景包括单用户端到端耗时、连续 NL 提交、Worker 长稳运行、`live-frame` 压力及多角色并发登录，用于验证非功能需求是否满足设计及发现资源瓶颈。

- **单用户端到端**：记录从提交渲染到 `completed` 总耗时与单帧渲染时间；
- **连续 NL**：连续 5 条不同指令，观察 `generate-json` 耗时与队列顺序；
- **长稳**：Worker 运行 30 分钟，监控内存与心跳是否中断；
- **预览压力**：20 FPS `live-frame` 持续 5 分钟，观察前端 CPU 与网络带宽；
- **并发登录**：三角色同时登录不同浏览器 profile，验证菜单隔离。

执行上述场景时，建议同时打开 API 控制台日志与 Worker 窗口输出，便于关联「前端感知延迟」与「后端/Blender 实际耗时」。若 `live-frame` 在 20 FPS 下导致前端 CPU 占

用偏高，可将 VITE_REALTIME_RENDER_FPS 与 BLENDER_LIVE_FPS 同步调低，在流畅度与资源之间取得平衡。

健康检查与 NL 解析响应均满足表 6 目标；单帧渲染时间随场景复杂度在可接受范围内波动；Worker 长稳运行期间心跳未中断。若需更高预览帧率，可通过 BLENDER_LIVE_FPS 调整，但应同步评估本机 CPU 与网络负载。

四、 项目管理

4.1 参与人员及分工

本项目采用「模块负责人 + 契约协同」的组织方式：前端、API/LLM、Blender 插件、测试各由明确分工的组员主责，同时通过 `shared/contracts/` 约定跨模块数据格式，减少联调歧义。经理负责里程碑、文档与验收组织；测试同学参与验收清单制定与 `tests/evidence/` 证据留存，使缺陷分级与修复优先级有统一标准。

表 四、.1: 项目团队分工

分工	姓名	学号	主要职责
经理	吴红祥	2312558	项目统筹、里程碑把控、文档与验收组织
前端	周昊川	2310126	Web 主控制台：登录/RBAC、三角色工作台、LlmRenderPanel
前端	吴行健	2310686	Web 主控制台：实时预览、路由与页面联调
后端（Blender 插件）	朱晨瑞	2312674	Blender icity 插件：场景生成与 NL 命令执行
后端（Blender 插件）	王旭	2312166	runtime_worker、渲染输出与插件集成
后端（LLM）	岳科言	2312163	自然语言解析：规则引擎、command.schema、LLM 接口
测试	谭易	2312260	功能与回归测试、验收清单、tests/evidence 证据留存

各分工与仓库模块对应关系：前端 → `web/apps/main-console`；Blender 插件 → `blender-plugin/`；LLM 解析 → `services/api (inferCommandsFromPrompt)`；测试 → `tests/`、`docs/test-plan.md`。分工边界以 `shared` 契约为界，接口变更须同步更新契约、测试计划与本报告相关章节，避免联调时「各模块自治、集成不一致」。

4.2 项目进展记录

项目自 2026 年 5 月下旬启动，按表 2 的 P0-P2 阶段与 `icity` 插件五阶段时间线并行推进；联调以「Web → API(`render_bridge`) → `runtime_worker` → WebSocket 预览」为主链路每日冒烟。下表汇总各阶段实际交付与当前状态，内容均可在仓库目录与 `docs/`、`blender-plugin/addons/icity/团队作业/` 中对照查阅。

表 四、.2: 项目进展记录

阶段	时间	完成内容	状态
P0 骨架搭建	5 月下旬	monorepo 目录拆分; shared/contracts/ 命令/模板/RBAC 契约; scripts/deploy.ps1、start.ps1 一键部署; docs/architecture.md、docs/test-plan.md 基线文档	已完成
P1 核心链路	5 月底-6 月初	API 登录与 RBAC、插件审核、需求/验收、规则 NL 解析 (inferCommandsFromPrompt); 内嵌 render_bridge 任务队列与 WebSocket; Web 三角色页面与 LlmRenderPanel; runtime_worker.py 心跳、拉任务、960×540 渲染与 live-frame 回传	已完成
icity 阶段一-二	5/23-5/31	插件可安装启用; 基础城市生成 (道路/建筑/树木); 道路纹理与路灯 3D 资产接入; templates.json 模板 0/1 差异化应用 (M1、M2)	已完成
icity 阶段三	6/01-6/06	path_animation.py 路径动画; scene_enhance.py 中央山水/水体; 车辆/行人/船只动态元素沿环路运动 (A7、A8 验收)	已完成
icity 阶段四	6/07-6/10	command_executor 与 NL handlers; ≥5 条中文指令离线稳定执行; 布局控制 (点线/草图); 与 API 规则引擎及 LLM 后端联调 (M4)	已完成
P2 联调收口	6/11-6/14	icity 能力整合至生产 Worker; 同源 Blender 打开 (open-linked-blender); 10 项核心验收清单全部通过; NL 回归集 7+ 条; 报告 LaTeX 源稿与编译脚本	已完成

联调与问题处理: P1 阶段曾出现 Blender 路径配置错误导致任务长期停留在 queued、

无 live-frame 更新的问题,通过在 `scripts/start.ps1` 中配置本机 `$blenderExe` 解决。NL 解析初期 `runtime_worker` 命令执行较薄,后复用 `icity command_executor` 完整实现,使 Web 提交的中文指令与 Blender 侧场景变更一致。测试同学按 `docs/test-plan.md` 制定 P0/P1 缺陷分级,联调阶段发现的问题均已修复;验收证据目录结构见 `tests/README.md`。

当前边界与后续规划: 业务数据仍采用 API 内存存储 (`store.js`),满足课程演示与反复验收,持久化数据库列为后续工作;NL 解析当前以 API 内嵌规则引擎为主,独立 `nlp-parser` 服务与自动化测试套件尚未落地。上述项不影响当前版本 10 项核心验收与 Windows 本机一键演示。

4.3 项目管理工具

版本与协作方面,团队使用 Git 管理代码;跨模块接口约定集中于 `shared/contracts/`; `scripts/deploy.ps1` 与 `start.ps1` 负责依赖安装与一键启动,降低演示环境重现成本;架构与测试说明分别维护于 `docs/architecture.md`、`docs/test-plan.md`。

五、 用户手册

用户手册面向首次部署与演示的操作人员,按部署启动、三角色典型工作流、Blender 插件安装与故障排查组织。若仅阅读前文架构章节,可从此处按步骤实际操作以验证理解;遇到预览异常或任务不推进时,优先对照「Blender 路径配置」与「故障排查」两节。

部署与启动详解

用户手册假定读者已克隆仓库并具备 Node.js 与 Blender 运行环境。整体流程为:先 `deploy.ps1` 安装依赖,再配置 Blender 路径,最后 `start.ps1` 一键拉起 API、Web 与 Worker。下列各节按实际操作顺序展开;若仅需快速验证 Web 登录与菜单,可只启动 API 与 Web,但 NL 渲染与实时预览必须保证 Worker 在线。

一键部署 (`deploy.ps1`)

在项目根目录执行 `./scripts/deploy.ps1` (或双击 `deploy.bat`)。脚本流程:

1. 检查 `node`、`npm` 是否在 `PATH`;
2. 若 `web/.env` 不存在则从 `.env.example` 复制;
3. 依次 `npm install`: `services/api`、`web/apps/main-console`。

Blender 路径配置

编辑 scripts/start.ps1 第 9 行附近:

```
$blenderExe = "D:\tools\blender\blender.exe"
```

常见路径示例:

- C:/Program Files/Blender Foundation/Blender 4.2/blender.exe
- D:/Apps/Blender/blender.exe

路径错误时 Worker 不会启动, 前端任务永久 queued、无 live-frame。

一键启动 (start.ps1)

1. 检测 API /health, 未运行则新窗口启动 `npm run dev` (API);
2. 等待 API 健康 (最长 60s);
3. 新窗口启动 Web (设置 `VITE_REALTIME_RENDER_FPS`);
4. 若 blender.exe 与 runtime_worker.py 存在, 后台启动 Blender Worker (设置 `API_BASE_URL`、`BLENDER_LIVE_FPS` 等环境变量)。

默认地址: Web <http://localhost:5173>, API <http://localhost:3001>。

系统管理员详细操作

1. 使用 admin / admin123 登录, 进入「系统首页」查看用户总数、插件状态、日志条数;
2. 「插件审核」: 查看 Smart City Generator v1.0, 点击「通过」或「驳回」, 状态写入 plugin 单例与 logs;
3. 「系统日志」: 按时间查看 plugin_review、acceptance、llm_generate、render_apply 等类型;
4. 「角色权限管理」: 查看三用户角色表 (当前为前端演示, 修改不持久化)。

行业分析师详细操作

1. 「行业需求提交」: 填写标题 (如「智慧城市建模核心能力」) 与描述, 提交后出现在需求列表;
2. 「功能验收」: 逐项勾选道路/建筑/资产/模板/NL/动态/生态, 点击保存; ≥ 5 项为「通过」;

3. 「评审记录」：查看历史 checkedItems、conclusion、reviewer、时间；
4. 「渲染结果」：输入「将天色变暗」→ 生成 JSON → 提交渲染 → 观察预览（无打开 Blender 按钮）。

场景建模师详细操作

1. 「工作台首页」：查看插件审核状态与快捷链接；
2. 「模板库」：阅读模板 0/1 的 tree_type、road_texture、building_density、weather 等 JSON 字段；
3. 「当前任务需求」：查看建模师任务列表页面；
4. 「渲染结果」：完整 NL 渲染流程 + Worker 信息（workerId、blendPath、进度）；
5. 点击「打开当前实时来源的 Blender」：确认 Worker 在线后拉起交互式 Blender，继续手动编辑。

Blender icity 插件安装与面板

安装 icity：

1. 将 blender-plugin/addons/icity 复制到 Blender 用户目录下的 scripts/addons；
2. Blender → 编辑 → 偏好设置 → 插件，搜索并启用 icity；
3. 按 N 打开侧边栏面板，先执行 Start ICity，再生成基础城市。

面板常用操作：生成基础城市 → Start ICity → 应用模板 → 执行 NL → 添加动态元素 → 渲染图片。

表 五、.1: 故障排查

现象	可能原因	处理
任务 queued 不动	Worker 未启动	检查 start.ps1 输出窗口
预览为 SVG 非照片	真实帧未回传	确认 Worker 日志无异常
WebSocket 断开	job 已完成或 API 重启	重新提交任务
401 未登录	Token 过期	重新登录
403 无权限	角色不符	换对应账号
icity 命令失败	未 Start ICity	先 scg.start_icity

故障排查表覆盖联调中最常见的五类现象。一般建议按「Worker 是否在线 → Token 是否有效 → 角色权限是否匹配 → icity 是否已 Start」的顺序排查；多数 queued 不动或预览仅为 SVG 占位的情况，均与 Worker 未启动或 Blender 路径配置错误有关。

5.1 总结与展望

SmartCityGen 智慧城市生成系统 面向智慧城市与数字孪生场景，实现了 Web 三角色协作、中文自然语言驱动、Blender 程序化生成与实时预览的一体化演示平台。项目按 P0-P2 阶段递进：先建立仓库骨架与共享契约，再打通渲染桥接与 Worker 主链路，最后整合 icity 插件能力并以测试与文档收口。当前版本 10 项核心验收均已通过，NL 回归集、渲染状态机与 live-frame 预览可在 Windows 本机通过一键脚本复现。

从软件工程视角，本项目的价值不仅在于「能生成城市场景」，更在于**跨技术栈协作模式**的实践：前端 React、Node.js API、Blender Python 与 JSON Schema 契约形成清晰边界；RBAC 与菜单下发保证多角色同平台不同视图；异步渲染桥接避免 Web 与三维引擎的紧耦合。上述模式可迁移至其他「Web 管控 + 重型计算 Worker」类系统。

后续可在不改变契约的前提下扩展：持久化数据库与真实 JWT、接入外部 LLM 替换规则引擎、多 Worker 负载均衡、更高分辨率与视频序列输出、以及基于 layout 图像的自动布点等。无论功能如何演进，`shared/contracts/` 作为单一真相源的设计应继续保持，以确保 Web、API 与 Blender 三端语义一致。